

SecureShield Report

Name	Abdelkarem Ahmed
Student number	220208715

Project Overview

SecureShield is a small Flask backend security demo that implements role-based access control using two roles:

user and admin. The application allows public registration and login, then protects selected routes using JWT authentication. Normal users can access their profile, while only admins can delete users.

The project also includes a simple browser testing console. This page is not the main security feature; it is a practical interface for demonstrating the API without manually copying JWT tokens through PowerShell commands. The backend remains a Flask API, and the web page calls the same API endpoints used in command-line testing.

Salting and Rainbow Table Protection

Passwords must not be stored in plain text because a database leak would immediately reveal every user password. Hashing is safer, but a simple unsalted hash can still be attacked with rainbow tables. A rainbow table is a precomputed list of common passwords and their matching hash values. If the same password always produces the same hash, attackers can compare leaked hashes against these tables.

Salting prevents this problem by adding a unique random value to each password before hashing. In Secure Shield, bcrypt handles this process automatically. The salt is generated and stored as part of the bcrypt hash, so two users with the same password still receive different stored hashes. This makes precomputed rainbow tables much less useful because attackers would need to recompute guesses for each unique salt.

Sensitive Data in JWT Payloads

JWT payloads are encoded, not encrypted. Anyone who has a JWT can decode the payload and read its claims. For this reason, sensitive information such as passwords, password hashes, private keys, security answers, or personal secrets should never be placed inside a JWT payload.

Secure Shield stores only the user id, username, role, issued-at time, and expiration time inside the token. These values are useful for authentication and authorization decisions, but they are not secrets. The token is still protected by a signature. If a user changes the role from user to admin without signing the token again with the real secret key, the server rejects the token because the signature no longer matches.

Least Privilege and RBAC

The Principle of Least Privilege means each user should only have the permission needed for their role. This reduces risk if a normal account is misused or compromised.

Secure Shield applies this principle using Role-Based Access Control. The project has two roles: user and admin. Both roles can access GET /profile, but only the admin role can access DELETE /user/. If a normal user tries to call the delete route, the API returns 403 Forbidden and records the attempt in security.log. If the token is missing, expired,

Token Blacklisting and Security Logging

JWTs are normally stateless, which means a valid token can continue to work until it expires. Secure Shield adds a logout feature by placing logged-out tokens into a blacklist. After logout, reusing the same token returns 401 Unauthorized, which demonstrates token revocation before natural expiry.

The application also writes unauthorized and forbidden attempts to security.log. This includes invalid tokens, blacklisted tokens, forbidden admin actions, and tampered JWT attempts. Logging these events helps demonstrate defensive monitoring and makes suspicious access attempts easier to review.

Conclusion

SecureShield successfully demonstrates bcrypt password hashing, JWT authentication, protected routes, role-based authorization, logout blacklisting, token tamper rejection, and defensive security logging. It is designed as an educational security demo that clearly shows the difference between normal user access and admin-only permissions.